



Department Department of Electrical & Electronic Engineering
Rajshahi University of Engineering & Technology

LAB Report

Course Code : **EEE-3100**

Course Title : **Electronics Shop Practice**

Experiment No. : 09

Experiment Name : Microcontroller Interfacing of Dual-Axis Joystick Module.

Submitted By:

Name : Labib Marwan Hoque

Roll : 2101064

Section : B

Submitted To:

Asif Zaman Rizve

Lecturer

Department of Electrical & Electronic
Engineering

Rajshahi University of Engineering & Technology

Date of Experiment : 16 June,2025

Date of Submission : 23 June,2025

Experiment No.: 09

Experiment Name: Microcontroller Interfacing of Dual-Axis Joystick Module.

Objective: -

- To interface a dual-axis analog joystick module with an Arduino microcontroller.
- To read and process analog signals from the joystick's X and Y axes.
- To understand the working principle of potentiometer-based joystick modules.
- To display the joystick position values in real-time via the Arduino serial monitor.
- To learn how to convert analog input signals into usable digital data for controlling devices or applications.

Theory:

A dual-axis joystick is an input device that provides two degrees of freedom movement along the X (horizontal) and Y (vertical) axes. It consists of two potentiometers arranged perpendicularly. Each potentiometer changes its resistance depending on the joystick's position along its respective axis.

- When the joystick is moved left or right, the X-axis potentiometer varies its resistance and produces a varying voltage signal.
- Similarly, moving the joystick forward or backward changes the resistance of the Y-axis potentiometer, generating another variable voltage.

These analog voltage signals (typically ranging from 0V to 5V) are fed into the Arduino's analog-to-digital converter (ADC) channels (analog input pins). The Arduino converts these analog signals into digital values (usually from 0 to 1023 for a 10-bit ADC).

By reading these digital values, the Arduino can determine the exact position of the joystick along both axes. This data can be used for various applications like controlling motors, navigating menus, or gaming controls.

Additionally, the joystick module often includes a push-button feature when pressed down, which can be read via a digital input pin.

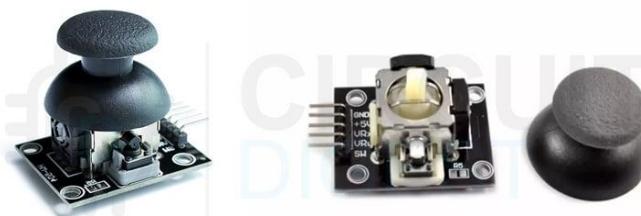


Fig.09.1: Dual Axis Joystick Module

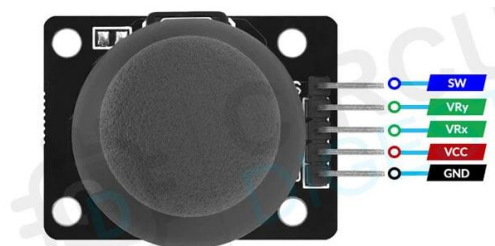


Fig.09.2: Pin Diagram of Dual Axis Joystick Module



Fig.09.3: Parts of Dual Axis Joystick Module

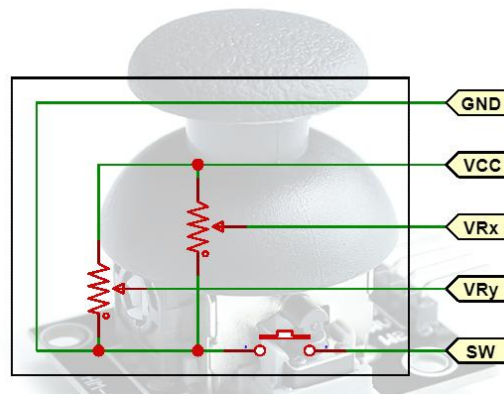


Fig.09.4: Circuit Diagram of Dual Axis Joystick Module

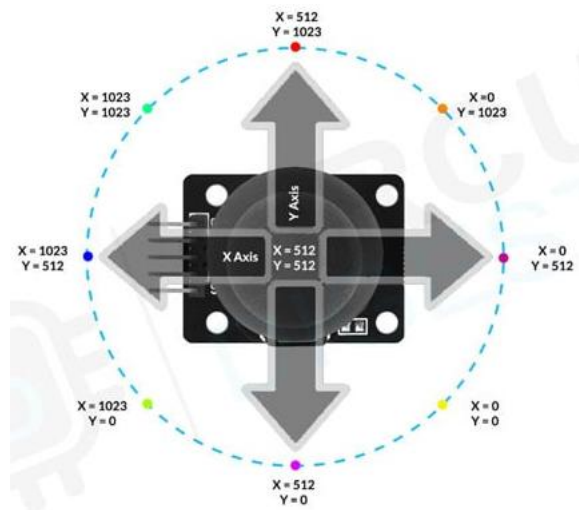


Fig.09.5: Equivalent Analog Value of Dual Axis Joystick Module

Required Apparatus:**Table 9.1: Table For Required Apparatus**

Sl. No.	Components	Specification	Quantity
1.	Arduino Uno	• ATmega328P • Operating Voltage: 5 V • Input Voltage: 7–12 V • Input Voltage: 6–20 V • Digital I/O Pins: 14 • PWM Digital I/O Pins: 6 (Pins 3, 5, 6, 9, 10, 11) • Analog Input Pins: 6 (A0 to A5) • DC Current per I/O Pin: 20 mA • DC Current for 3.3 V Pin: 50 mA • Flash Memory: 32 KB (ATmega328P) of which 0.5 KB used by bootloader • SRAM: 2 KB (ATmega328P) • EEPROM: 1 KB (ATmega328P) • Clock Speed: 16 MHz • USB Connection: Type B • LED Built-in: Pin 13	1
2.	LED	Red, Green, Blue, Yellow	4
3.	Resistors	1 K Ω	4
4.	Dual Axis Joystick Module	Operating Voltage – 5V	1
5.	DC Power	15V	-
6.	Jumper Wires	-	-
7.	IDE Software	• Arduino IDE	-

Circuit Diagram:

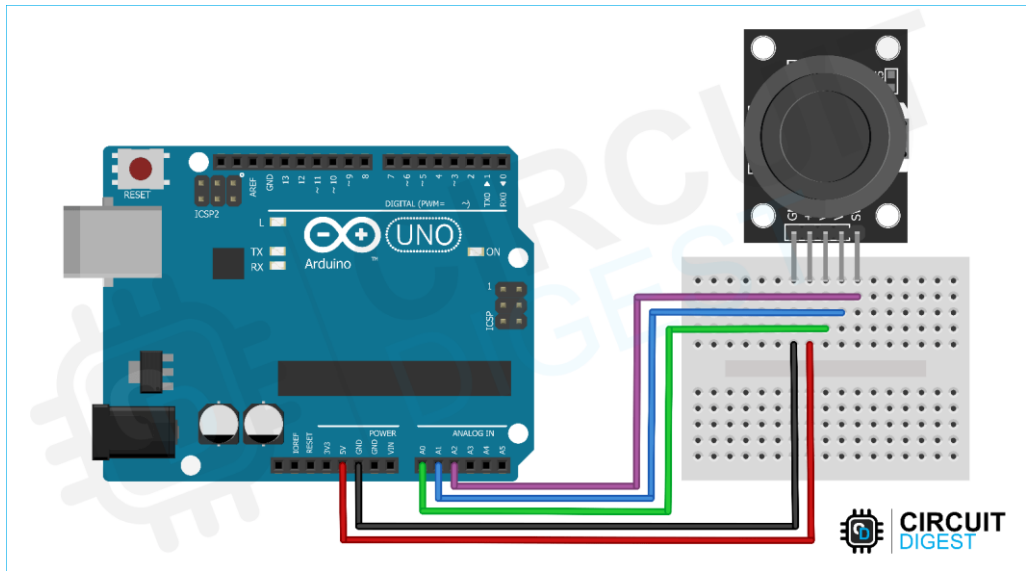


Fig.09.6: Circuit Diagram of Dual Axis Joystick Module

Experimental Setup:

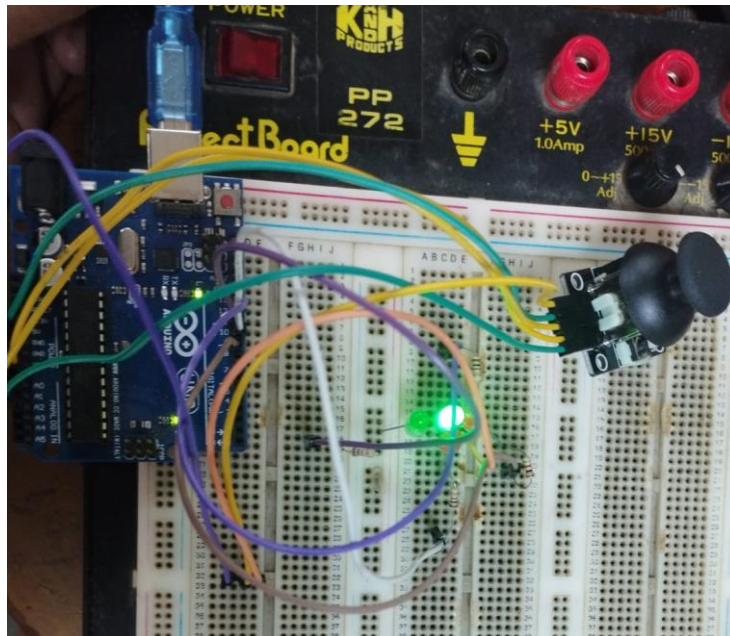


Fig.09.7: Experimental Setup of the Dual Axis Joystick Module

Code:

<pre>// Arduino Dual Axis Joystick Example #define Xaxis_pin A0 // Arduino pin connected to the VRx Pin #define Yaxis_pin A1 // Arduino pin connected to the VRy Pin #define SW_pin A2 // Arduino pin connected to the SW Pin void setup() { pinMode(SW_pin, INPUT); digitalWrite(SW_pin, HIGH); Serial.begin(9600); }</pre>	<pre>void loop() { Serial.print("X-axis: "); Serial.print(analogRead(Xaxis_pin)); Serial.print(" : "); Serial.print("Y-axis: "); Serial.print(analogRead(Yaxis_pin)); Serial.print(" : "); Serial.print("Switch: "); Serial.println(digitalRead(SW_pin)); delay(200); }</pre>
--	--

Experimental Procedure:

1. The Arduino Uno board was placed on a breadboard or work surface to prepare for wiring.
2. The dual-axis joystick module was connected to the Arduino as follows:
 - The VCC pin of the joystick was connected to the 5V pin of the Arduino.
 - The GND pin of the joystick was connected to the GND pin of the Arduino.
 - The VRx pin (X-axis) was connected to analog pin A0 of the Arduino.
 - The VRy pin (Y-axis) was connected to analog pin A1 of the Arduino.
 - The SW pin (push-button) was connected to digital pin D2 of the Arduino.
3. A USB cable was used to connect the Arduino Uno to the computer.
4. The Arduino IDE was opened, and the provided source code was copied into a new sketch.
5. The correct board (Arduino Uno) and COM port were selected from the Tools menu.
6. The code was compiled and uploaded to the Arduino Uno board using the Upload button.
7. The Serial Monitor was opened from the Arduino IDE and the baud rate was set to 9600 bps.
8. The joystick was moved in different directions (left, right, up, down, center), and the corresponding messages were observed on the Serial Monitor.
9. The joystick was also pressed down to test the push-button functionality, and the message "Joystick Button Pressed" was confirmed on the Serial Monitor.
10. The experiment was repeated multiple times to verify accurate readings from all directions and the button.

Task No.: 01

Task Name: Directional Input-Based LED Indication System Using Joystick.

Objectives:

- To design and implement a real-time directional indication system using a dual-axis analog joystick and Arduino.
- To detect directional movements (left, right, up, and down) from the joystick and activate corresponding LEDs for visual feedback.
- To demonstrate analog input processing and digital output control using embedded system principles.
- To ensure accurate mapping of physical joystick movements to specific LED indicators, verifying responsiveness and intuitive control behavior.

Code:

<pre>#define Xaxis_pin A0 // Joystick VRx #define Yaxis_pin A1 // Joystick VRy #define SW_pin A2 // Joystick Switch #define LED1 10 // LED for UP #define LED2 9 // LED for DOWN unsigned long led1StartTime = 0; unsigned long led2StartTime = 0; bool led1Active = false; bool led2Active = false; void setup() { pinMode(SW_pin, INPUT); digitalWrite(SW_pin, HIGH); pinMode(LED1, OUTPUT); pinMode(LED2, OUTPUT); Serial.begin(9600); } void loop() { int yValue = analogRead(Yaxis_pin); int xValue = analogRead(Xaxis_pin); int swState = digitalRead(SW_pin); Serial.print("X-axis: "); Serial.print(xValue); Serial.print(" : Y-axis: "); Serial.print(yValue); Serial.print(" : Switch: "); Serial.println(swState); // Joystick UP if (yValue > 800) { digitalWrite(LED1, HIGH); digitalWrite(LED2, LOW); if (!led1Active) { led1StartTime = millis(); led1Active = true; led2Active = false; } }</pre>	<pre>Serial.print("LED1 ON Time: "); Serial.print((millis() - led1StartTime) / 1000.0); Serial.println(" seconds"); } // Joystick DOWN else if (yValue < 200) { digitalWrite(LED1, LOW); digitalWrite(LED2, HIGH); if (!led2Active) { led2StartTime = millis(); led2Active = true; led1Active = false; } Serial.print("LED2 ON Time: "); Serial.print((millis() - led2StartTime) / 1000.0); Serial.println(" seconds"); } // Joystick Neutral else { digitalWrite(LED1, LOW); digitalWrite(LED2, LOW); led1Active = false; led2Active = false; } delay(2000); }</pre>
--	---

Task No.: 02

Task Name: Joystick-Based Directional Reflex Testing System

Objectives:

- To develop an interactive reflex testing system using a joystick and Arduino, where LEDs light up randomly to simulate directional prompts.
- To detect user response by matching joystick direction with the randomly activated LED and turning the LED off upon correct movement.
- To measure and display the user's reaction time from LED activation to correct joystick input, demonstrating real-time input handling and decision logic.
- To enhance understanding of random signal generation, analog direction sensing, and timer-based performance evaluation in embedded systems.

Code:

<pre>#define Xaxis_pin A0 #define Yaxis_pin A1 #define SW_pin A2 #define UP_LED 11 #define DOWN_LED 9 #define LEFT_LED 8 #define RIGHT_LED 10 int randVal; unsigned long startTime, endTime; bool ledOn = false; unsigned long lastStopwatchPrint = 0; void setup() { pinMode(SW_pin, INPUT); digitalWrite(SW_pin, HIGH); pinMode(UP_LED, OUTPUT); pinMode(DOWN_LED, OUTPUT); pinMode(LEFT_LED, OUTPUT); pinMode(RIGHT_LED, OUTPUT); Serial.begin(9600); randomSeed(analogRead(0)); } void loop() { int xVal = analogRead(Xaxis_pin); int yVal = analogRead(Yaxis_pin); if (!ledOn) { digitalWrite(UP_LED, LOW); digitalWrite(DOWN_LED, LOW); digitalWrite(LEFT_LED, LOW); digitalWrite(RIGHT_LED, LOW); randVal = random(4); startTime = millis(); lastStopwatchPrint = 0; switch (randVal) { case 0: digitalWrite(UP_LED, HIGH); Serial.println("LED ON: UP"); break; case 1:</pre>	<pre> if (ledOn && millis() - lastStopwatchPrint >= 500) { float liveTime = (millis() - startTime) / 1000.0; Serial.print("Elapsed Time: "); Serial.print(liveTime, 2); Serial.println(" seconds"); lastStopwatchPrint = millis(); } bool correctMove = false; switch (randVal) { case 0: if (yVal > 800) correctMove = true; break; // UP case 1: if (yVal < 200) correctMove = true; break; // DOWN case 2: if (xVal > 800) correctMove = true; break; // RIGHT swapped case 3: if (xVal < 200) correctMove = true; break; // LEFT swapped } if (correctMove) { digitalWrite(UP_LED, LOW); digitalWrite(DOWN_LED, LOW); digitalWrite(LEFT_LED, LOW); digitalWrite(RIGHT_LED, LOW); endTime = millis(); float duration = (endTime - startTime) / 1000.0; switch (randVal) { case 0: Serial.print("UP LED"); break;</pre>
---	---

<pre> digitalWrite(DOWN_LED, HIGH); Serial.println("LED ON: DOWN"); break; case 2: digitalWrite(LEFT_LED, HIGH); Serial.println("LED ON: RIGHT"); // Swapped message break; case 3: digitalWrite(RIGHT_LED, HIGH); Serial.println("LED ON: LEFT"); break; } ledOn = true; } </pre>	<pre> case 1: Serial.print("DOWN LED"); break; case 2: Serial.print("RIGHT LED"); break; // Swapped message case 3: Serial.print("LEFT LED"); break; // Swapped message } Serial.print(" OFF. Final Time Taken: "); Serial.print(duration, 2); Serial.println(" seconds\n"); delay(1000); ledOn = false; } delay(50); } </pre>
--	--

Results:

Task 01: Direction-Based LED Control Using Joystick

In this task, the joystick was moved in different directions, and we observed that the corresponding LEDs turned on correctly. When the joystick was pushed upward, the UP LED turned on. When moved downward, the DOWN LED turned on. The center position turned both LEDs off. The system worked smoothly and responded properly to the joystick's movement. This confirmed that the joystick's analog signals were being read correctly by the Arduino, and the LEDs were controlled as expected.

Task 02: Reaction Timer Using Joystick and Random LED Challenge

In this task, an LED lit up randomly in one direction (UP, DOWN, LEFT, or RIGHT), and we had to quickly move the joystick in the correct direction to turn it off. The time taken to respond was shown in seconds on the Serial Monitor. We noticed that the reaction time changed depending on how fast we moved the joystick. Each time a new LED turned on, the timer started, and after giving the correct input, the final time was displayed. This showed that the system could accurately measure and display human response time using the joystick.

Experimental Result:

From Serial Monitor:

```

Output: LED ON: LEFT
Elapsed Time: 0.00 seconds
Elapsed Time: 0.50 seconds LEFT LED OFF. Final Time Taken: 0.75 seconds
LED ON: UP
Elapsed Time: 0.00 seconds UP LED OFF. Final Time Taken: 0.15 seconds
LED ON: UP
Elapsed Time: 0.00 seconds
UP LED OFF. Final Time Taken: 0.00 seconds

```

Discussions:

This experiment showed how we can use joystick movements to control devices and measure reaction times. By reading signals from the joystick and triggering LEDs, we built a small system that responds like a game controller—quick, interactive, and fun to test.

Justification of CO2 and PO(j):

We chose the joystick module because it gives analog signals that clearly match directional motion. With Arduino, we converted those signals into real actions—like lighting LEDs and tracking how fast a person reacts. That proves how embedded systems can combine hardware and software for responsive control.

Conclusion:

This experiment helped us see how a joystick can be used for accurate control and real-time feedback. It gave us practical experience with analog signals, testing reflexes, and building basic embedded systems.

Reference:

<https://circuitdigest.com/microcontroller-projects/microcontroller-interfacing-dual-axis-joystick-module-with-arduino>